

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <module type="JAVA_MODULE" version="4">
3   <component name="NewModuleRootManager" inherit-
    compiler-output="true">
4     <exclude-output />
5     <content url="file://$MODULE_DIR$">
6       <sourceFolder url="file://$MODULE_DIR$/src"
        isTestSource="false" />
7     </content>
8     <orderEntry type="inheritedJdk" />
9     <orderEntry type="sourceFolder" forTests="false"
    />
10  </component>
11 </module>
```

```
1 ### IntelliJ IDEA ###
2 out/
3 !**/src/main/**/out/
4 !**/src/test/**/out/
5 .kotlin
6
7 ### Eclipse ###
8 .apt_generated
9 .classpath
10 .factorypath
11 .project
12 .settings
13 .springBeans
14 .sts4-cache
15 bin/
16 !**/src/main/**/bin/
17 !**/src/test/**/bin/
18
19 ### NetBeans ###
20 /nbproject/private/
21 /nbbuild/
22 /dist/
23 /nbdist/
24 /.nb-gradle/
25
26 ### VS Code ###
27 .vscode/
28
29 ### Mac OS ###
30 .DS_Store
```

```
1 import adapter.LegacyBankApi;
2 import adapter.LegacyBankApiAdapter;
3 import adapter.PaymentProcessor;
4 import builder.Computer;
5 import builder.ComputerDirector;
6 import builder.GamingComputerBuilder;
7 import builder.OfficeComputerBuilder;
8 import chain.FirstLineSupport;
9 import chain.SupportHandler;
10 import chain.SupportManager;
11 import chain.SupportRequest;
12 import chain.TechnicalSupport;
13 import decorator.EmailNotifier;
14 import decorator.Notifier;
15 import decorator.SmsNotifierDecorator;
16 import decorator.TelegramNotifierDecorator;
17 import proxy.Image;
18 import proxy.ImageProxy;
19 import strategy.BubbleSortStrategy;
20 import strategy.MergeSortStrategy;
21 import strategy.QuickSortStrategy;
22 import strategy.Sorter;
23
24 import java.util.Arrays;
25
26 public class Main {
27     public static void main(String[] args) {
28         strategyDemo();
29         chainDemo();
30         builderDemo();
31         proxyDemo();
32         decoratorDemo();
33         adapterDemo();
34     }
35
36     private static void strategyDemo() {
37         System.out.println("=== Стратегия ===");
38         int[] data = {5, 3, 8, 1, 9, 2};
39         Sorter sorter = new Sorter(new
BubbleSortStrategy());
40         System.out.println("Исходник:      " + Arrays
```

```
40 .toString(data));
41     System.out.println("BubbleSort:    " + sorter
    .sortAsString(data));
42     sorter.setStrategy(new QuickSortStrategy());
43     System.out.println("QuickSort:      " + sorter
    .sortAsString(data));
44     sorter.setStrategy(new MergeSortStrategy());
45     System.out.println("MergeSort:      " + sorter
    .sortAsString(data));
46     System.out.println("Исходник не изменился: "
    + Arrays.toString(data));
47     System.out.println();
48 }
49
50 private static void chainDemo() {
51     System.out.println("=== Цепочка обязанностей
    ===");
52     SupportHandler firstLine = new
    FirstLineSupport();
53     firstLine.setNext(new TechnicalSupport()).
    setNext(new SupportManager());
54     firstLine.handle(new SupportRequest(1, "Как
    сменить пароль?"));
55     firstLine.handle(new SupportRequest(2, "Не
    запускается приложение"));
56     firstLine.handle(new SupportRequest(3, "Прошу
    вернуть деньги за подписку"));
57     firstLine.handle(new SupportRequest(4, "
    Известный запрос"));
58     System.out.println();
59 }
60
61 private static void builderDemo() {
62     System.out.println("=== Строитель ===");
63     ComputerDirector director = new
    ComputerDirector(new OfficeComputerBuilder());
64     System.out.println(director.buildOfficePc());
65     director.setBuilder(new GamingComputerBuilder
    ());
66     System.out.println(director.buildGamingPc());
67     System.out.println("Ручная сборка: " + new
```

```
67 GamingComputerBuilder()
68     .withCpu("Intel Core Ultra 9")
69     .withRam(64)
70     .build());
71     System.out.println();
72 }
73
74     private static void proxyDemo() {
75         System.out.println("=== Прокси ===");
76         Image image = new ImageProxy("photo.png", "
admin");
77         System.out.println("Первый вызов:");
78         image.display();
79         System.out.println("Второй вызов:");
80         image.display();
81         System.out.println("Вызов без прав:");
82         new ImageProxy("photo.png", "guest").display
();
83         System.out.println();
84     }
85
86     private static void decoratorDemo() {
87         System.out.println("=== Декоратор ===");
88         Notifier notifier = new EmailNotifier("user@
mail.ru");
89         System.out.println("Только e-mail:");
90         notifier.send("Сервис будет недоступен 15
минут");
91
92         System.out.println("E-mail + SMS + Telegram
:");
93         Notifier rich = new
TelegramNotifierDecorator(
94             new SmsNotifierDecorator(notifier,
"+79990001122"),
95             "user");
96         rich.send("Оплата прошла успешно");
97         System.out.println();
98     }
99
100     private static void adapterDemo() {
```

```
101         System.out.println("=== Адаптер ===");
102         PaymentProcessor processor = new
        LegacyBankApiAdapter(new LegacyBankApi());
103         System.out.println("USD 2500 центов: " +
        processor.pay("ACC-1", 2500, "USD"));
104         System.out.println("RUB 100000 копеек: " +
        processor.pay("ACC-1", 100000, "RUB"));
105         System.out.println("JPY 5000: " + processor.
        pay("ACC-1", 5000, "JPY"));
106     }
107 }
108
```

```
1 package chain;
2
3 public abstract class SupportHandler {
4     private SupportHandler next;
5
6     public SupportHandler setNext(SupportHandler next
7 ) {
8         this.next = next;
9         return next;
10    }
11
12    public void handle(SupportRequest request) {
13        if (canHandle(request)) {
14            process(request);
15        } else if (next != null) {
16            next.handle(request);
17        } else {
18            System.out.println("[Поддержка] Никто не
19 обработал запрос: " + request.getMessage());
20        }
21    }
22
23    protected abstract boolean canHandle(
24    SupportRequest request);
25
26    protected abstract void process(SupportRequest
27    request);
28 }
```

```
1 package chain;
2
3 public class SupportManager extends SupportHandler {
4     @Override
5     protected boolean canHandle(SupportRequest
6         request) {
7         return request.getUrgency() == 3;
8     }
9     @Override
10    protected void process(SupportRequest request) {
11        System.out.println("[Менеджер] Принял решение
12        по запросу: " + request.getMessage());
13    }
14 }
```

```
1 package chain;
2
3 public class SupportRequest {
4     private final int urgency;
5     private final String message;
6
7     public SupportRequest(int urgency, String message
8 ) {
9         this.urgency = urgency;
10        this.message = message;
11    }
12
13    public int getUrgency() {
14        return urgency;
15    }
16
17    public String getMessage() {
18        return message;
19    }
20 }
```

```
1 package chain;
2
3 public class FirstLineSupport extends SupportHandler
4 {
5     @Override
6     protected boolean canHandle(SupportRequest
7     request) {
8         return request.getUrgency() == 1;
9     }
10    @Override
11    protected void process(SupportRequest request) {
12        System.out.println("[1-я линия] Ответили на
13        вопрос: " + request.getMessage());
14    }
15 }
```

```
1 package chain;
2
3 public class TechnicalSupport extends SupportHandler
4 {
5     @Override
6     protected boolean canHandle(SupportRequest
7     request) {
8         return request.getUrgency() == 2;
9     }
10
11     @Override
12     protected void process(SupportRequest request) {
13         System.out.println("[Техподдержка] Исправили
14         проблему: " + request.getMessage());
15     }
16 }
```

```
1 package proxy;  
2  
3 public interface Image {  
4     void display();  
5  
6     String getFileName();  
7 }  
8
```

```
1 package proxy;
2
3 public class RealImage implements Image {
4     private final String fileName;
5
6     public RealImage(String fileName) {
7         this.fileName = fileName;
8         loadFromDisk();
9     }
10
11     private void loadFromDisk() {
12         System.out.println("  Загрузка файла с диска
13 : " + fileName);
14     }
15
16     @Override
17     public void display() {
18         System.out.println("  Отображение изображения
19 : " + fileName);
20     }
21
22     @Override
23     public String getFileName() {
24         return fileName;
25     }
26 }
```

```
1 package proxy;
2
3 public class ImageProxy implements Image {
4     private final String fileName;
5     private final String allowedRole;
6     private RealImage realImage;
7
8     public ImageProxy(String fileName, String
allowedRole) {
9         this.fileName = fileName;
10        this.allowedRole = allowedRole;
11    }
12
13    @Override
14    public void display() {
15        if (!"admin".equals(allowedRole)) {
16            System.out.println(" [Proxy] Доступ
запрещён для роли: " + allowedRole);
17            return;
18        }
19        if (realImage == null) {
20            System.out.println(" [Proxy] Первое
обращение – создаю реальный объект");
21            realImage = new RealImage(fileName);
22        } else {
23            System.out.println(" [Proxy] Изображение
взято из кеша: " + fileName);
24        }
25        realImage.display();
26    }
27
28    @Override
29    public String getFileName() {
30        return fileName;
31    }
32 }
33
```

```
1 package adapter;
2
3 public class LegacyBankApi {
4     public int charge(String accountId, double
    amountInEuros) {
5         if (amountInEuros <= 0) {
6             return 1;
7         }
8         System.out.println("  LegacyBankApi: списание
    " + String.format("%.2f", amountInEuros)
9             + " EUR со счёта " + accountId);
10        return 0;
11    }
12 }
13
```

```
1 package adapter;  
2  
3 public interface PaymentProcessor {  
4     boolean pay(String accountId, int amountCents,  
5         String currency);  
6 }
```

```
1 package adapter;
2
3 import java.util.Map;
4
5 public class LegacyBankApiAdapter implements
    PaymentProcessor {
6     private static final Map<String, Double>
    RATES_TO_EUR = Map.of(
7         "EUR", 1.0,
8         "USD", 0.92,
9         "RUB", 0.0095
10    );
11
12    private final LegacyBankApi legacyApi;
13
14    public LegacyBankApiAdapter(LegacyBankApi
    legacyApi) {
15        this.legacyApi = legacyApi;
16    }
17
18    @Override
19    public boolean pay(String accountId, int
    amountCents, String currency) {
20        Double rate = RATES_TO_EUR.get(currency);
21        if (rate == null) {
22            System.out.println(" [Adapter]
    Неподдерживаемая валюта: " + currency);
23            return false;
24        }
25        double amountInEuros = Math.round(amountCents
    * rate) / 100.0;
26        return legacyApi.charge(accountId,
    amountInEuros) == 0;
27    }
28 }
29
```

```
1 package builder;
2
3 public class Computer {
4     private final String cpu;
5     private final int ramGb;
6     private final int storageGb;
7     private final String gpu;
8     private final String os;
9
10    public Computer(String cpu, int ramGb, int
storageGb, String gpu, String os) {
11        this.cpu = cpu;
12        this.ramGb = ramGb;
13        this.storageGb = storageGb;
14        this.gpu = gpu;
15        this.os = os;
16    }
17
18    public String getCpu() {
19        return cpu;
20    }
21
22    public int getRamGb() {
23        return ramGb;
24    }
25
26    public int getStorageGb() {
27        return storageGb;
28    }
29
30    public String getGpu() {
31        return gpu;
32    }
33
34    public String getOs() {
35        return os;
36    }
37
38    @Override
39    public String toString() {
40        return "Computer{cpu='" + cpu + "', ram=" +
```

```
40 ramGb + "GB, storage=" + storageGb
41       + "GB, gpu='" + gpu + "', os='" + os
   + "'}";
42     }
43 }
44
```

```
1 package builder;
2
3 public interface ComputerBuilder {
4     ComputerBuilder withCpu(String cpu);
5
6     ComputerBuilder withRam(int ramGb);
7
8     ComputerBuilder withStorage(int storageGb);
9
10    ComputerBuilder withGpu(String gpu);
11
12    ComputerBuilder withOs(String os);
13
14    Computer build();
15 }
16
```

```
1 package builder;
2
3 public class ComputerDirector {
4     private ComputerBuilder builder;
5
6     public ComputerDirector(ComputerBuilder builder
7 ) {
8         this.builder = builder;
9     }
10
11     public void setBuilder(ComputerBuilder builder) {
12         this.builder = builder;
13     }
14
15     public Computer buildOfficePc() {
16         return builder
17             .withCpu("Intel Core i5-14400")
18             .withRam(16)
19             .withStorage(512)
20             .build();
21     }
22
23     public Computer buildGamingPc() {
24         return builder
25             .withCpu("AMD Ryzen 9 9950X")
26             .withRam(64)
27             .withStorage(4000)
28             .build();
29     }
30 }
```

```
1 package builder;
2
3 public class GamingComputerBuilder implements
  ComputerBuilder {
4     private String cpu;
5     private int ramGb;
6     private int storageGb;
7     private String gpu;
8     private String os;
9
10    public GamingComputerBuilder() {
11        reset();
12    }
13
14    private void reset() {
15        cpu = null;
16        ramGb = 32;
17        storageGb = 2000;
18        gpu = "RTX 5090";
19        os = "Windows 11";
20    }
21
22    @Override
23    public ComputerBuilder withCpu(String cpu) {
24        this.cpu = cpu;
25        return this;
26    }
27
28    @Override
29    public ComputerBuilder withRam(int ramGb) {
30        this.ramGb = ramGb;
31        return this;
32    }
33
34    @Override
35    public ComputerBuilder withStorage(int storageGb
36    ) {
37        this.storageGb = storageGb;
38        return this;
39    }
```

```
40     @Override
41     public ComputerBuilder withGpu(String gpu) {
42         this.gpu = gpu;
43         return this;
44     }
45
46     @Override
47     public ComputerBuilder withOs(String os) {
48         this.os = os;
49         return this;
50     }
51
52     @Override
53     public Computer build() {
54         if (cpu == null) {
55             throw new IllegalStateException("Для
игрового ПК не задан процессор");
56         }
57         if (ramGb < 16) {
58             throw new IllegalStateException("Для
игрового ПК нужно минимум 16 ГБ RAM");
59         }
60         Computer computer = new Computer(cpu, ramGb,
storageGb, gpu, os);
61         reset();
62         return computer;
63     }
64 }
65
```

```
1 package builder;
2
3 public class OfficeComputerBuilder implements
  ComputerBuilder {
4     private String cpu;
5     private int ramGb;
6     private int storageGb;
7     private String gpu;
8     private String os;
9
10    public OfficeComputerBuilder() {
11        reset();
12    }
13
14    private void reset() {
15        cpu = null;
16        ramGb = 8;
17        storageGb = 256;
18        gpu = "Встроенная графика";
19        os = "Linux";
20    }
21
22    @Override
23    public ComputerBuilder withCpu(String cpu) {
24        this.cpu = cpu;
25        return this;
26    }
27
28    @Override
29    public ComputerBuilder withRam(int ramGb) {
30        this.ramGb = ramGb;
31        return this;
32    }
33
34    @Override
35    public ComputerBuilder withStorage(int storageGb
36    ) {
37        this.storageGb = storageGb;
38        return this;
39    }
```

```
40     @Override
41     public ComputerBuilder withGpu(String gpu) {
42         this.gpu = gpu;
43         return this;
44     }
45
46     @Override
47     public ComputerBuilder withOs(String os) {
48         this.os = os;
49         return this;
50     }
51
52     @Override
53     public Computer build() {
54         if (cpu == null) {
55             throw new IllegalStateException("Для
офисного ПК не задан процессор");
56         }
57         if (ramGb < 4) {
58             throw new IllegalStateException("Для
офисного ПК нужно минимум 4 ГБ RAM");
59         }
60         Computer computer = new Computer(cpu, ramGb,
storageGb, gpu, os);
61         reset();
62         return computer;
63     }
64 }
65
```

```
1 package strategy;
2
3 import java.util.Arrays;
4
5 public class Sorter {
6     private SortStrategy strategy;
7
8     public Sorter(SortStrategy strategy) {
9         this.strategy = strategy;
10    }
11
12    public void setStrategy(SortStrategy strategy) {
13        this.strategy = strategy;
14    }
15
16    public int[] sort(int[] data) {
17        return strategy.sort(data);
18    }
19
20    public String sortAsString(int[] data) {
21        return Arrays.toString(sort(data));
22    }
23 }
24
```

```
1 package strategy;  
2  
3 public interface SortStrategy {  
4     int[] sort(int[] data);  
5 }  
6
```

```
1 package strategy;
2
3 public class MergeSortStrategy implements
  SortStrategy {
4     @Override
5     public int[] sort(int[] data) {
6         int[] arr = data.clone();
7         mergeSort(arr, arr.length);
8         return arr;
9     }
10
11     private void mergeSort(int[] arr, int n) {
12         if (n < 2) {
13             return;
14         }
15         int mid = n / 2;
16         int[] left = new int[mid];
17         int[] right = new int[n - mid];
18         System.arraycopy(arr, 0, left, 0, mid);
19         System.arraycopy(arr, mid, right, 0, n - mid
20 );
21         mergeSort(left, mid);
22         mergeSort(right, n - mid);
23         merge(arr, left, right);
24
25     private void merge(int[] arr, int[] left, int[]
26 right) {
27         int i = 0;
28         int j = 0;
29         int k = 0;
30         while (i < left.length && j < right.length) {
31             if (left[i] <= right[j]) {
32                 arr[k++] = left[i++];
33             } else {
34                 arr[k++] = right[j++];
35             }
36         }
37         while (i < left.length) {
38             arr[k++] = left[i++];
39         }
40     }
41 }
```

```
39         while (j < right.length) {  
40             arr[k++] = right[j++];  
41         }  
42     }  
43 }  
44
```

```
1 package strategy;
2
3 public class QuickSortStrategy implements
  SortStrategy {
4     @Override
5     public int[] sort(int[] data) {
6         int[] arr = data.clone();
7         quickSort(arr, 0, arr.length - 1);
8         return arr;
9     }
10
11     private void quickSort(int[] arr, int left, int
  right) {
12         if (left >= right) {
13             return;
14         }
15         int pivot = arr[(left + right) / 2];
16         int i = left;
17         int j = right;
18         while (i <= j) {
19             while (arr[i] < pivot) {
20                 i++;
21             }
22             while (arr[j] > pivot) {
23                 j--;
24             }
25             if (i <= j) {
26                 int tmp = arr[i];
27                 arr[i] = arr[j];
28                 arr[j] = tmp;
29                 i++;
30                 j--;
31             }
32         }
33         quickSort(arr, left, j);
34         quickSort(arr, i, right);
35     }
36 }
37
```

```
1 package strategy;
2
3 public class BubbleSortStrategy implements
  SortStrategy {
4     @Override
5     public int[] sort(int[] data) {
6         int[] arr = data.clone();
7         for (int i = 0; i < arr.length - 1; i++) {
8             for (int j = 0; j < arr.length - 1 - i; j
9                 ++) {
10                 if (arr[j] > arr[j + 1]) {
11                     int tmp = arr[j];
12                     arr[j] = arr[j + 1];
13                     arr[j + 1] = tmp;
14                 }
15             }
16         }
17         return arr;
18     }
19 }
```

```
1 package decorator;  
2  
3 public interface Notifier {  
4     void send(String message);  
5 }  
6
```

```
1 package decorator;
2
3 public class EmailNotifier implements Notifier {
4     private final String email;
5
6     public EmailNotifier(String email) {
7         this.email = email;
8     }
9
10    @Override
11    public void send(String message) {
12        System.out.println(" E-mail на " + email +
13        ": " + message);
14    }
15 }
```

```
1 package decorator;
2
3 public abstract class NotifierDecorator implements
  Notifier {
4     protected final Notifier wrapped;
5
6     protected NotifierDecorator(Notifier wrapped) {
7         this.wrapped = wrapped;
8     }
9 }
10
```

```
1 package decorator;
2
3 public class SmsNotifierDecorator extends
  NotifierDecorator {
4     private final String phone;
5
6     public SmsNotifierDecorator(Notifier wrapped,
  String phone) {
7         super(wrapped);
8         this.phone = phone;
9     }
10
11     @Override
12     public void send(String message) {
13         wrapped.send(message);
14         System.out.println(" SMS Ha " + phone + ": "
  + message);
15     }
16 }
17
```

```
1 package decorator;
2
3 public class TelegramNotifierDecorator extends
  NotifierDecorator {
4     private final String username;
5
6     public TelegramNotifierDecorator(Notifier wrapped
  , String username) {
7         super(wrapped);
8         this.username = username;
9     }
10
11     @Override
12     public void send(String message) {
13         wrapped.send(message);
14         System.out.println(" Telegram @" + username
  + ": " + message);
15     }
16 }
17
```